TIETO PUBLIC 360 · INTEGRATIONSGUIDE

Opsætning af 360° API-authentikering

FK Adgangsstyring Service Accounts — fra bootstrap til drift

Denne guide beskriver, hvordan du sætter authentikering op fra bunden til 360° API'et via **FK Adgangsstyring Service Accounts**, og hvordan det bruges sammen med `src/public360_client.py` (klassen `Public360Client`) i dette repo.

Kilde: `360_API_Auth_FK_Adgangsstyring_CustIntGuide.pdf` (Tieto, Customer Integration Guide).

1. Overblik

Authentikeringen sker i to faser, begge via OAuth 2.0 mod Keycloak (som ligger på 360°-infrastrukturen og delegerer login til FK Adgangsstyring via SAML):

FASE	HVORNÅR	FLOW	RESULTAT
1. Bootstrap	Én gang (manuelt)	OAuth 2.0 Device Authorization Grant + PKCE	Et **offline token** (langtidsholdbart refresh token med `offline_access` scope)
2. Runtime	Ved hvert API-kald (automatisk)	OAuth 2.0 Refresh Token Grant	Et kortlivet **access token** (~10 min), sendt som `Authorization: Bearer <token>`

Offline-tokenet er en hemmelighed på niveau med et kodeord: det sendes **aldrig** til selve API'et, det logges aldrig, og det opbevares i jeres sikre secret-store.

2. Ansvarsfordeling

Tieto gør:

- Opretter en Keycloak bootstrap-klient til jeres integration
- Udleverer jeres `realm-url` og `client-id`
- Kan levere en bootstrap-CLI og sample-kode, hvis det ønskes

I (kunden) gør:

- Kører bootstrap **én gang** for at hente et offline token
- Opbevarer og beskytter offline-tokenet i jeres secret store

- Implementerer runtime-klienten (token exchange + API-kald) — dette er allerede gjort i `public360_client.py`
- Håndterer token-fornyelse/gendannelse ved driftsfejl

3. Forudsætninger — hvad skal indhentes, og hvorfra

Fra Tieto (jeres 360°-kontakt) — bed om dette, før I starter:

VÆRDI	FORMAT/EKSEMPEL	BRUGES TIL
`realm-url`	`https://keycloak.360online.com/realms/<customer-realm>`	Alle OAuth-kald (discovery, token, revoke)
`client-id`	Fri tekst/GUID leveret af Tieto	Identificerer jeres integration i Keycloak

Når I beder om oprettelse af klienten, skal I til gengæld oplyse Tieto:

- Jeres 360-tenant hostname, fx `kommune.public360online.com`
- Formål/type på integrationen (fx "nattlig datasync", "sagsopslag")
- Hvis I skal bruge flere klienter: et unikt navn/ID pr. klient

Selv besluttet (lokalt):

VÆRDI	EKSEMPEL	BEMÆRKNING
`tenant_host`	`kommune.public360online.com`	Basis-URL for selve 360° API-kaldene
`token_storage_path`	`./fk-api-offline-token.local.json`	Hvor offline-tokenet gemmes. Brug jeres godkendte secret store (Key Vault, environment-variabel osv.) i produktion — en lokal fil er kun et udgangspunkt

Der er **intet brugernavn/kodeord** at konfigurere i koden. Det eneste login, der finder sted, er ét interaktivt browser-login under bootstrap (trin 5), foretaget af en FK-servicebruger.

4. Forbered miljøet

```
pip install requests
```

`src/public360_client.py` kræver ingen andre afhængigheder.

5. Kør bootstrap (engangs, kræver en person ved tasterne)

Bootstrap skal køres af en operatør, der kan logge ind via FK Adgangsstyring med den relevante FK-servicebruger.

```
from src.public360_client import Public360Client

client = Public360Client(
    realm_url="https://keycloak.360online.com/realms/<customer-realm>",   # fra
Tieto
    client_id="<bootstrap-client-id>",                                     # fra
Tieto
    tenant_host="<kommune>.public360online.com",                           # jeres
tenant
    token_storage_path="./fk-api-offline-token.local.json",               # jeres
secret store
)

client.bootstrap()
```

Hvad der sker, når `bootstrap()` kaldes:

1. Klienten henter Keycloaks OpenID-konfiguration fra `{realm-url}/.well-known/openid-configuration`.
2. Der genereres et PKCE code-verifier/code-challenge-par.
3. Klienten beder Keycloak om en device-autorisation og får en `verification_uri` tilbage.
4. URL'en printes i konsollen og åbnes automatisk i en browser (kan slås fra med `bootstrap(open_browser=False)`).
5. I browseren logger I ind via FK Adgangsstyring (SAML) med FK-servicebrugeren.
6. Klienten poller Keycloaks token-endpoint i baggrunden, indtil login er gennemført.
7. Det modtagne offline token (refresh token) gemmes i `token_storage_path` med filrettigheder `600` (kun ejeren kan læse/skrive filen).

Efter dette trin er bootstrap **færdig** — det skal ikke køres igen, medmindre tokenet bliver ugyldigt (se afsnit 8).

⚠️ **Flyt filen til jeres rigtige secret store** (Key Vault, Vault, krypteret config osv.) efter bootstrap — den lokale JSON-fil er kun et transportmedie, ikke den anbefalede permanente opbevaring.

6. Brug klienten i runtime (automatisk, ingen login)

Når offline-tokenet er gemt, kræver alle efterfølgende kald ingen menneskelig interaktion:

```
from src.public360_client import Public360Client

client = Public360Client(
    realm_url="https://keycloak.360online.com/realms/<customer-realm>",
    client_id="<bootstrap-client-id>",
    tenant_host="<kommune>.public360online.com",
    token_storage_path="./fk-api-offline-token.local.json",
)

response = client.get("/some/api/path")
print(response.json())
```

Ved hvert kald sker der automatisk:

1. Offline-tokenet indlæses fra `token_storage_path`.
2. Det udveksles til et kortlivet access-token via `grant_type=refresh_token` mod Keycloaks token-endpoint.
3. Hvis Keycloak roterer refresh-tokenet, gemmes den nye værdi automatisk.
4. Access-tokenet caches i hukommelsen og genbruges, indtil det er ved at udløbe.
5. API-kaldet foretages med `Authorization: Bearer <access_token>`.
6. Fejl håndteres efter guidens baseline:
   - **401** → tokenet fornyes tvunget, og kaldet forsøges igen én gang.
   - **429** → `Retry-After`-headeren respekteres, med backoff.
   - **5xx** → eksponentiel backoff, op til 5 forsøg.

7. Automatiseret drift — robot/script uden manuel login

Dette er det punkt, mange forveksler: **bootstrap er ikke noget robotten gør**. Det er et engangs-login, foretaget af et menneske, som producerer en fil robotten derefter genbruger for evigt (eller indtil den bliver ugyldig). De to roller bør leve i hver sit script:

SCRIPT	KØRES AF	HVOR OFTE	KRÆVER LOGIN?
`scripts/bootstrap_once.py`	Et menneske, interaktivt, med en browser	Én gang (og igen hvis tokenet bliver ugyldigt)	Ja — via FK Adgangsstyring
`scripts/robot_job.py`	Cron / scheduled task / service — ubemandet	Så ofte jobbet skal køre (hver nat, hver time osv.)	Nej — genbruger den gemte token-fil

Fremgangsmåde

1. **Kør bootstrap manuelt, én gang**, på en maskine hvor du kan åbne en browser:

   ```
   python scripts/bootstrap_once.py
   ```

   Dette logger dig ind via FK Adgangsstyring og gemmer offline-tokenet i en fil.

2. **Flyt/deploy token-filen** til det miljø robotten skal køre i (server, container, secret store). Robotten og bootstrap-scriptet skal pege på **samme** `token_storage_path`.

3. **Lad robotten køre automatisk** — den initialiserer blot klienten og kalder `client.get(...)` / `client.post(...)`. Der sker ingen browser-interaktion; klienten udveksler stille og roligt det gemte offline token til et frisk access-token ved hvert kald:

   ```
   python scripts/robot_job.py
   ```

   Sæt dette op i jeres scheduler (cron, Task Scheduler, Kubernetes CronJob osv.) som et helt almindeligt ubemandet job.

Hvis robotten fejler med en authentikeringsfejl

Hvis offline-tokenet er blevet ugyldigt (tilbagekaldt, udløbet, eller FK-rettigheder ændret), kaster klienten en `Public360AuthError`. Der findes **ingen automatisk vej videre** herfra — det er selve sikkerhedspointet med FK Adgangsstyring: et nyt offline token kræver et nyt menneskeligt login. Robotten bør derfor:

- fange `Public360AuthError` eksplicit (se `scripts/robot_job.py`)
- sende en alarm/notifikation til drift eller en administrator
- **ikke** forsøge at gen-authentikere sig selv

En administrator kører herefter `scripts/bootstrap_once.py` igen, og robotten virker igen fra næste kørsel.

8. Sikkerhedsregler (skal overholdes)

- Behandl offline-tokenet som en hemmelighed på linje med et kodeord.
- Gem det aldrig i kildekode, plaintext-configs, logs, tickets eller screenshots.
- Send det aldrig til 360° API'et — kun det korte access-token må sendes som Bearer-token.
- Brug separate klienter/tokens pr. integration, hvis muligt.

9. Hvornår skal I re-bootstrappe?

Kør `python scripts/bootstrap_once.py` (eller `client.bootstrap()`) igen, hvis:

- Token-exchange fejler med `invalid_grant` (tokenet er ugyldigt, udløbet eller tilbagekaldt) — klienten kaster i så fald en `Public360AuthError` med denne besked.
- Tokenet er blevet tilbagekaldt (se afsnit 10).
- FK-servicebrugerens rettigheder er ændret, og ændringen skal slå igennem med det samme.
- I mistænker, at tokenet er lækket eller kompromitteret.

10. Tilbagekald tokenet (revocation)

Brug dette ved mistanke om lækage, nedlukning af en integration, eller hvis rettighedsændringer skal have øjeblikkelig effekt:

```
client.revoke()
```

Efter tilbagekaldelse er tokenet ugyldigt med det samme, og der **skal** køres `bootstrap()` igen, før klienten kan bruges videre.

11. Fejlfinding (fra guiden)

SYMPTOM	ÅRSAG	LØSNING
`invalid_grant` ved token-exchange	Token er tilbagekaldt/udløbet, eller client-id passer ikke	Kør `bootstrap()` igen
Intet `refresh_token` i svaret	`offline_access` scope er ikke tilladt for klienten	Kontakt Tieto og bekræft klientens scopes
401/403 med gyldigt token	Rettighedsmapping eller audience/scope stemmer ikke	Bekræft FK-servicebrugerens rettigheder med Tieto

12. Reference: hvilke Keycloak-endpoints bruges

Alle hentes dynamisk fra `{realm-url}/.well-known/openid-configuration` (mere robust ved serverændringer end at hardcode dem):

- `device_authorization_endpoint` — bruges i `bootstrap()`
- `token_endpoint` — bruges både i `bootstrap()` (polling) og runtime (refresh)
- `{realm-url}/protocol/openid-connect/revoke` — bruges i `revoke()`

Baseret på Tieto's "360° API Authentication via FK Adgangsstyring Service Accounts — Customer Integration Guide" og repoets `src/public360_client.py`.